

Spellbook Technical Report

Dewayne Scrivner, Mason Allen, Jesse Pickard, Jordan Weaver

CSCI 4805
Dr. Nicholson
May 6, 2026

Introduction

The project, Spellbook, is meant to help collectors of Magic: The Gathering (MTG) cards have an easily manageable and accessible system to store their cards. There are over 27,000 unique MTG cards, which makes them hard to track manually. Spellbook is a web app for logging MTG collections and managing decks. It allows users to add and remove cards from their inventory and decks, search for cards using various filters, and view different prints for cards.

This web app is targeted towards MTG players and collectors who want a centralized, accessible place to manage and store their cards. Organizing large amounts of cards requires significant space and time, and if users are looking for a specific card it can take a long time to find it. However, with Spellbook, users can easily see their cards, search for what is in their inventory, and check whether they own a card. This report will cover the technology, design, deploying/building, and current known issues of the website.

Technology

Spellbook was built using a JavaScript-based web stack, which combined several tools for frontend, backend, and deployment. The technologies used in this project are React, Next.js, Supabase, Vercel, Scryfall API, Visual Studio Code (VS Code), and GitHub.

React

React is a popular open-source JavaScript library used for building user interfaces (UIs), specifically for single-page applications where data changes dynamically without requiring a page reload. The team chose React because Spellbook is a web application, and React's component-based structure made it practical to build reusable UI pieces such as the Navbar and card grids. It helped simplify the creation of a clean, visually-pleasing UI and made it easier for our group to manage the visual appearance of the site. React was challenging to learn as none of us have ever used it before and we were required to learn CSS, learn state management, and understand component-based architecture. It didn't require too much time from our team to get comfortable with, but it was new and thus required some effort to use effectively.

NextJS

Next.js is an open-source web development framework built on top of React that enables the creation of high-performance, full-stack web applications. The role it played was used to enhance the frontend interface and backend logic as well. The framework made it easier for the app to route to other pages. It helped simplify things like tour navigation, page structure, and

seamless communication with client-side components and server data. Its optimization improves performance, and it is very compatible with Vercel. It made deployment from the site very easy. Learning it could be difficult, since you'll have to learn its syntax and how to use it properly. Client and server components, asynchronous data fetching, and dynamic routing are all features that require practice and time to become proficient with. Overall, Next.js significantly improves development and makes the process much easier. The prebuilt tool and libraries make implementing the feature easier for developers. Since they don't have to develop all these tools from scratch

Supabase

Supabase is an open-source, "Backend-as-a-Service" (BaaS) platform built on top of PostgreSQL. In Spellbook, Supabase was used for authentication, storing inventory and deck data, and handling database operations for our website. This included updating, querying, and inserting card information. It was our primary backend system, allowing the application to get user-specific data. The team chose Supabase since it was free, easy to integrate, and gave an easy way to use Google authentication. This gave us a way for users to log in and store data in one. Setting up Supabase was a challenge, but any issues were quickly resolved. The only real issue was that it made it hard to test out log in specific feature. Since it disabled npm run dev. A lot of the code for updating, inserting, and deleting was pretty straightforward once we got the hang of it. This benefits the project's development, as it allows us to offload all saved data to the cloud. If we had to physically store all of this data. It could add up quickly.

Vercel

Vercel is a cloud platform designed for frontend developers to easily deploy and host websites and web applications. For Spellbook, Vercel is used to deploy our application and make it easily accessible to users through a live, hosted environment. It provides continuous deployment by automatically updating the website in real time. So that changes are quickly accessible whenever they occur. It offers fast performance, easy deployment via our repository, and strong compatibility with NextJS. Overall, it benefited our development by simplifying the deployment process and enabling us to quickly see the impact of our changes to the application. The only real drawbacks were things like the domain name, but otherwise, it fit all our project needs.

Scryfall API

The Scryfall API is a REST-like web service that allows developers to programmatically access the extensive Scryfall Magic: The Gathering card database. Spellbook heavily relies on Scryfall as a source for all of the MTG cards rather than storing them in our own database. It also keeps our site up-to-date since Scryfall updates every twelve hours. Scryfall was chosen early on in planning, as the team recognized that storing all card data independently would be impractical and demand more resources than we had available. Scryfall was challenging for several of our team as some of us had never incorporated an API before, so it took some time to adjust and figure out how to integrate it into our systems and functions. We learned by relying on each other and information provided by documentation and online resources. Scryfall largely assisted our team in development, as it saved us from having to store all of the MTG card information and its endpoints are well-documented and straightforward to query.

Visual Studio Code

VS Code is a free, lightweight, and powerful open-source code editor developed by Microsoft for Windows, macOS, and Linux. VS Code played a significant part in the development of Spellbook by being a convenient coding tool that works seamlessly with GitHub. We chose VS Code specifically because it was convenient with the built-in GitHub integration, extensions, and live error detection. VS Code is not too different from other IDEs like IntelliJ or NetBeans, so it wasn't very challenging to navigate and learn. Our team has some experience with these other IDEs due to past coding classes, and it made the transition to VS Code straightforward. VS Code had a positive impact on the development of Spellbook due to its ability to run Spellbook locally via npm, with Next.js serving the app.

GitHub

GitHub is a cloud-based platform that acts as a digital hub for developers to create, store, manage, and share code. GitHub allowed our team to work remotely and work on different pieces of the code simultaneously, as well as keeping track of our changes and history. GitHub was required for our assignment, but it was a great tool for changing and updating code, and it was reliable to prevent conflicts when attempting to change the main branch. Our team had a small learning curve for GitHub in the beginning, but after the initial hurdle, it was easy to navigate and use. Unfortunately, our team was unable to fully utilize the branching system and primarily worked from the main branch directly. This didn't have a significant impact on our system, but it sometimes made working on code inconvenient or worrisome for fear of breaking something or conflicting with someone else's work. There were also a couple of merging issues where some code got deleted after large pushes, but these were solved relatively quickly and had no long-lasting impact on the application. Despite these hiccups, GitHub had a very positive impact on Spellbook and greatly assisted in its development. Without GitHub, coordinating changes across four team members working remotely would have been significantly more difficult.

Design

Frontend Components

Spellbook's frontend is built as a collection of reusable React components. Most of our frontend displays are housed in the `src/app/components/` folder in the Next.js project, and many of the JavaScript files have a corresponding CSS module for styling, with larger pages importing these components as necessary. Additionally, each component and page includes dark mode support via the `prefers-color-scheme` media, automatically adjusting colors based on the user's system preferences.

Navbar

The Navbar is a key component of Spellbook, and it is visible on every page. It navigates users from page to page using the navigation links, and there is also a search bar for cards. The Navbar also renders different elements depending on whether a user is logged in. For example, if a user has not been authenticated, they see the Login button and are navigated to the Login page when trying to go to the Inventory or Decks page. When users are logged in, they see their avatar

instead of a login button, and they can navigate pages freely. Visually, the Navbar has the logo image and text in the top left, then the three navigation links beneath it. The active navigation link has bold letters and a white box around it, and hovering over a navigation link will underline the word. To the right of that is the card search bar, which is shrunk on smaller screens into a clickable magnifying glass icon that opens the search bar as a popup. On the right of the Navbar is the login button or avatar depending on the user authentication state.

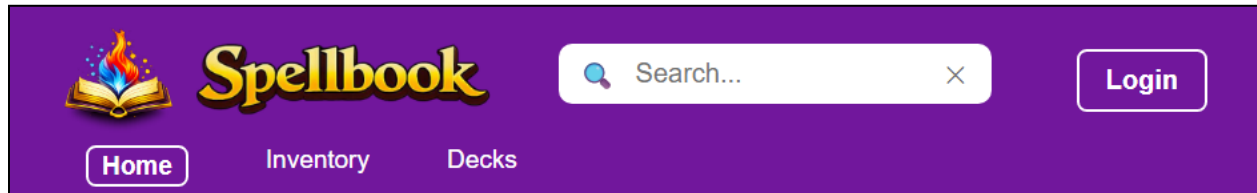


Image 1: Spellbook Navbar

The search bar queries the Scryfall API using an autocomplete endpoint that is triggered whenever the user enters input into the field. But once the user reaches the minimum length, a short delay is added to prevent excessive API calls. After this delay, cards with names matching the input are returned to the user. The results are stored in a state and displayed in a drop-down menu. To give a user-friendly UI and maintain good performance. The page only shows a limited number of results. Once the user finds their desired result, they can click the card, and it calls the API a second time to pull the card's id. This is used to link it to the card info page and displays the card's full details. There is also error handling to ensure a smooth experience in case the API is slow or fails.

The Navbar fetches the current user on load using Supabase's `getUser()` function and listens for authentication changes in real time using `onAuthStateChange`, ensuring the correct elements are displayed without requiring a page refresh. Login is handled through Google OAuth via Supabase's `signInWithOAuth`, which redirects users back to their original destination after authentication. Logging out calls Supabase's `signOut` function and redirects the user to the homepage.

The Navbar also manages the user's avatar selection and persistence. The Navbar uses a `useEffect` hook that reads the user's selected avatar from their Supabase metadata and passes it down to the `AvatarPicker` component as a prop. When a user selects a new avatar, the Navbar updates the local state immediately and calls Supabase's `updateUser` function to persist the selection to the user's metadata.

The `AvatarPicker` is a component that allows users to select an avatar for their account. This requires users to be logged in, and it displays when users click on the avatar symbol on the Navbar. Upon click, the Navbar displays a popup menu containing the user's email, a logout button, and the embedded `AvatarPicker` component which renders a 2x3 grid of avatar choices. When the user hovers over a clickable button in the popup menu, it will have a little background change as an indicator, and the currently selected avatar will have a constant purple background and outline. The `AvatarPicker` component receives `selected` and `onSelect` as props from the Navbar, which emphasizes the connection between these two components.

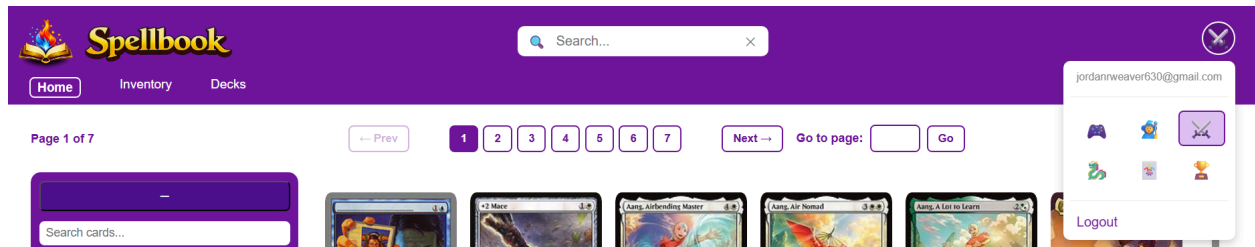


Image 2: Spellbook Navbar, showing profile icons and homepage features.

Homepage

The CollapsibleMenu is the filter sidebar visible on the left side of the CardGrid. It works to allow the users to sift through the vast collection of cards. The filters within the component are subcomponents, depending on what data each filter is expected to use. The only exception to a direct type relation is the color filter; it utilizes a custom four state checkbox. Each component takes a hook to control its state. The search bar within it can be used to directly utilize scryfall's syntax for more advanced searching. The remaining filter options work to build a query that is passed to our Scryfall API calls. The two types can work in conjunction with each other as both queries are sent to the API. The menu component raises the results via its props allowing other components like CardGrid to utilize the data in order to display images or sort through any of the card's data. Alongside the set of cards, the menu takes a pagination value from its parents so that upon triggering a search, the page count resets to one. On smaller devices the filters within the menu reorganize into a column to save space.

The CardGrid is a visual component displayed on Spellbook's Home and Inventory pages. It displays cards in a responsive image system, so the grid changes dynamically depending on screen size. For example, on larger screens, multiple cards are displayed per row, and on small screens, there may only be one card displayed per row. Cards are fetched from Scryfall via `scryfallApi.browse()` using a `useEffect` hook. Up to 81 cards are displayed per page, and each card is a clickable link that routes to `/CardInfo/${card.id}` using Next.js dynamic routing. Additionally, the collapsible filter menu, `CollapsibleMenu.js`, is imported and visible on the left side of the screen. There's a pagination component built inside this file called the `PaginationBar`, which displays numbered pages, prev/next buttons, ellipsis for large page counts (though the current fetch limit affects the total page count in practice), and a "Go to page" jump input for a given page. There is also a page indicator that displays what page the user is currently on out of the total pages available. The pagination bar and the page indicator are visible at the top and bottom of the screen. The pagination bar is in the center above and below the CardGrid, while the page indicator is top left and bottom right.

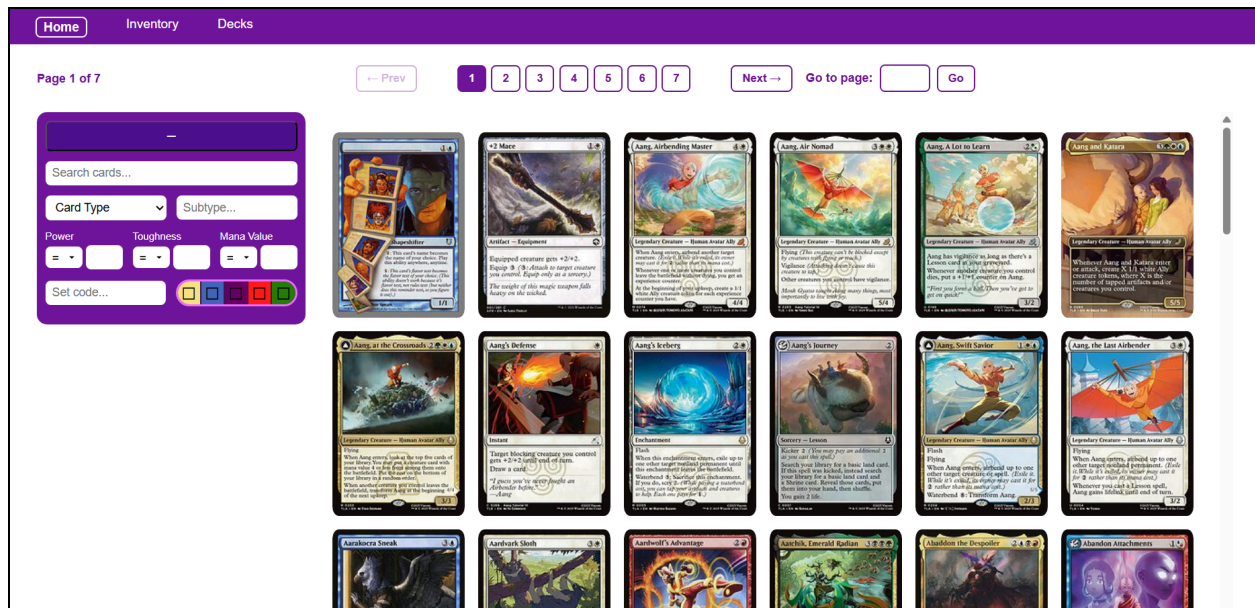


Image 3: Spellbook homepage

Card Information Page

The Card Info page displays detailed information for a specific card when it is clicked from the CardGrid or InventoryCardGrid, or when it is searched in the Navbar search bar. The primary purpose of the card info page is to display externally sourced card data, along with some user-specific data, to the system's database. The page includes dynamic features, such as tracking the number of cards the user owns. The page also uses client-side components with Next.js and React.

The page's structure is built around React components that handle data retrieval, state management, and page rendering. A wrapper component uses React's Suspense feature to provide a fallback to a loading state while the main content is being prepared. This ensures the user sees something instead of a blank page during load to improve UI design. When loaded, it displays an image of the card, card data, user's inventory quantity, symbols, alternate prints, etc. The page automatically updates itself for the inventory function.

The card's data is pulled from the Scryfall API. The information being pulled includes the card image, card name, card type, mana cost, oracle text, power, toughness, flavor text, set name, collector number, rarity, set type, artist, and card legality. The card's ID is retrieved and the corresponding information is displayed. The mana symbols for the page are processed and dynamically inserted. Since the icon cannot be pulled directly from the card's ID, images are assigned to the icons and replace the placeholders. The API sends placeholder values, for example, water mana is represented as {U}. The corresponding image from the Scryfall API is assigned to each of the given outputs. Occasionally, if the site is slow, the curly braces may still appear briefly. This improves readability, since most users may not know what the placeholders mean. Alternate prints are displayed in a scrollable grid. Lower-quality images are used to improve performance, especially on cards with many prints. All cards with the same ID are displayed, and when one is clicked, the user is brought to that page.

Some cards in MTG have two sides. In the API, they are stored as card faces rather than a single card image. Early on, this caused double-sided cards not to appear when their card info

page was opened. To correct this, the program checks if the card uses card faces. A flip button is provided that, when clicked, changes the card's image and the information for that card. Reprints for double-sided cards only show the card's front side.

For adding and removing cards from the user's inventory, functionality is implemented so that when the button is clicked, it either increments or decrements the card's value in the database. For the add function, the system checks if the user pressing the button is a valid user. Even though the buttons only appear when the user is logged in, there is still a possibility of errors, so validation is performed for safety. The system then checks whether the card already exists in the inventory table. If it does not, a row is created for that card and the necessary card data is added to the database. Then it checks if the card exists within the user's specific inventory. If it does not exist, a row is created for that user and the card's quantity is set to 1. If the card already exists for the user, the row is updated and the quantity is incremented. After this, the database is queried again to retrieve the updated quantity, which is then displayed to the user.

The remove function is similar to the add function but with fewer steps. It first checks if the user is authenticated. Then it checks the database to determine if the card exists. If it does not, no action is taken; if it does, one quantity is removed from the card in the inventory. If there are two or more copies, the quantity is decremented by one. If there is only one card in the user's inventory, the row containing that card is deleted.

To improve UI design, a loading screen with animation is implemented. This prevents the page from appearing blank while loading. The page is displayed only after the API data retrieval is successful, preventing UI issues caused by missing elements. This also provides protection against edge cases, such as the Scryfall API being unavailable. It increases stability and reduces the likelihood of runtime errors.

The page's design is responsive. The layout includes a large image alongside the card's information, with both sections maintaining similar sizing to ensure readability without excessive scrolling. When content exceeds the available space, the information section becomes scrollable. Reprints are placed at the bottom, based on the assumption that card details are more important to users than alternate versions. When the page width drops below 820px, the layout rearranges to improve responsiveness. All information is displayed vertically, which improves usability on mobile devices. In this layout, the scrollable feature within the card info section is removed to enhance mobile usability. Dark and light modes are also implemented to ensure readability based on user preference.

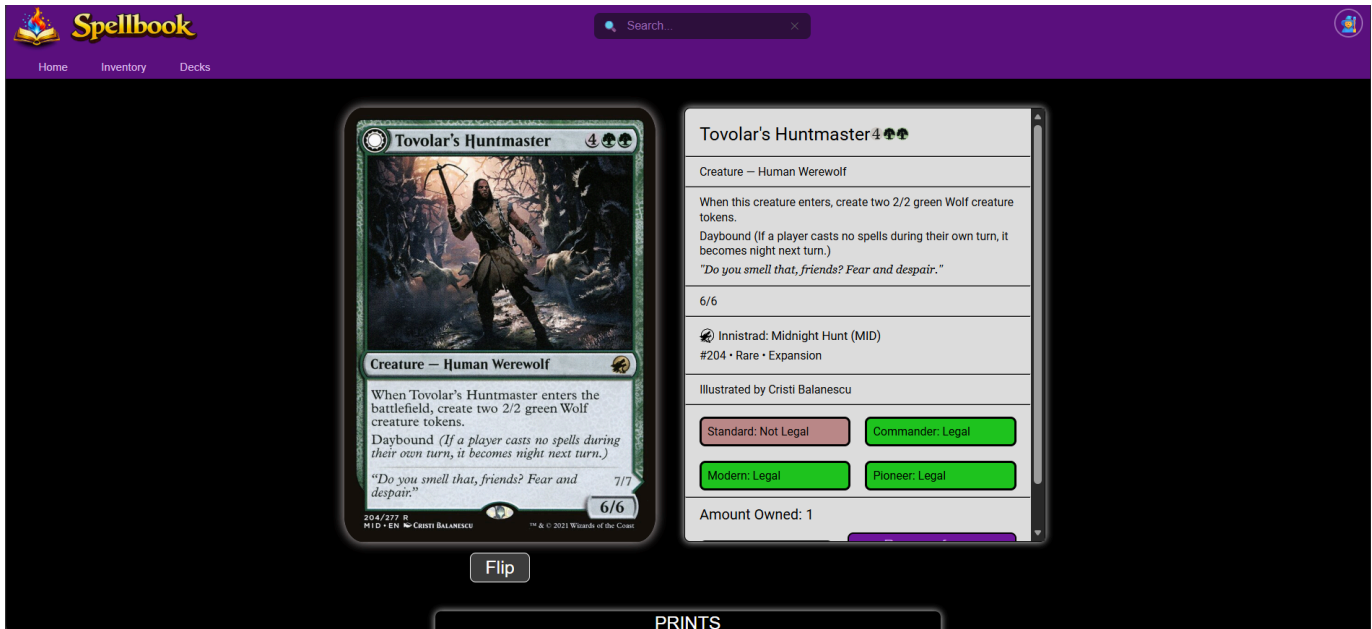


Image 4: Card info page for Trovolar's Huntmaster

Inventory Page

The InventoryCardGrid is a variant of the CardGrid that displays only cards in the user's inventory. It also retrieves, organizes, and displays all cards associated with an authenticated user. The page combined both client-side and server-side components to maximize user functionality. The card grid for the homepage gets all of its data directly from the Scryfall API. While the inventory card grid gets its information from the database and then fills in the rest from the API. To ensure that it only shows cards the user owns.

The grid displays works much like the search function. The main difference is the source of the data. If the user has enough cards, they can choose different pages of cards to look at. Since each page has a limit on the number of cards that can be viewed at once.

The Inventory filters work similarly to the search filters. The search filters worked by applying them and then searching for the card. This meant only the cards that applied showed up. For the inventory filters, all cards have already been pulled from the database. So the program quickly applies the filters and only shows on the screen what matches them. There's a compare function to check the operation of what is being compared for the mana cost, power, and toughness. Also, the color filters remained the same, except they applied to the inventory cards. For the inventory page, some functionality had to be rewritten to handle the conditionals for mana cost, power, and toughness. But they work the same. Same for the color, ID, and exclude functionality. Putting the inventory cards into the original function wasn't working. So the function had to be slightly altered to produce the correct output.

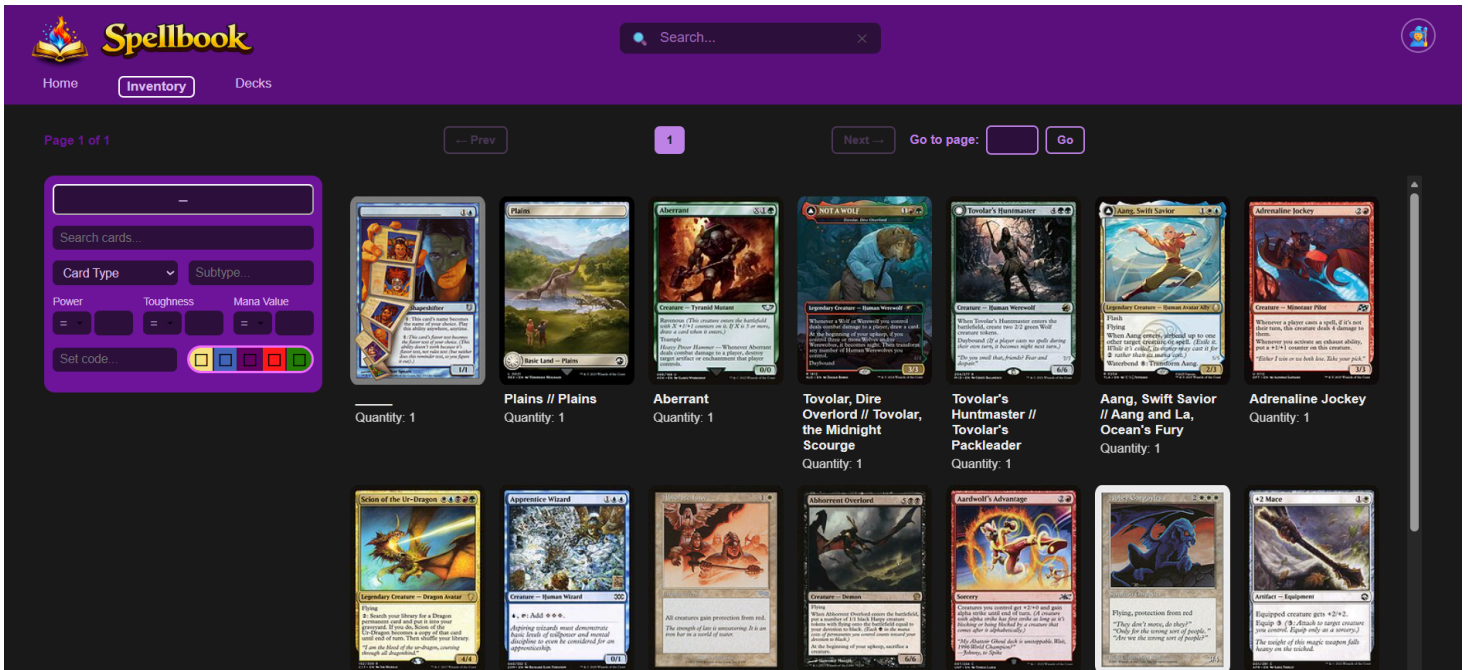


Image 5: Inventory page

Deck Management System

The DeckGrid is displayed when the user navigates to the Decks page using the Decks navigation link. The DeckTile component displays the user's decks as a grid of fanned card stacks with the deck name beneath. Additionally, if the deck has selected a format, the format will be displayed below the deck name. Each deck renders visually as three fanned cards with a left, middle, and right position. If the deck has a commander card, it always goes in the middle card slot. When hovering over a deck, there's a popup that shows the deck name, "Contains:" with up to five card names listed after, whether any of the named cards are missing from the user's inventory and how many, and "...and X more" for larger decks. The popup will also reposition depending on screen position to avoid going off-screen. There are also three purple placeholders of slightly different shades that will fill in the fanned cards when a deck has fewer than three cards.

There is a pagination for DeckGrid where a maximum of 20 decks can appear per page. The pagination bar is hidden when all decks fit on a single page. When there are only two pages, there is only a prev/next button, and then a full pagination for three or more pages. There are also handlers for loading and error states, where it shows "Loading your decks..." when fetching information, an error message when the fetch fails, and "You don't have any decks yet!" when the user has no decks.

The DeckGrid fetches information using Supabase. It fetches decks in order of creation date, so they are displayed from newest (top left) to oldest (bottom right). For every deck, card count is fetched from the deck_cards table. Then, the user's inventory is checked to identify missing cards, which are then flagged with a warning in the popup if they are in the first five cards fetched. Then, the fan card images are built by querying card names then fetching the images from Scryfall.

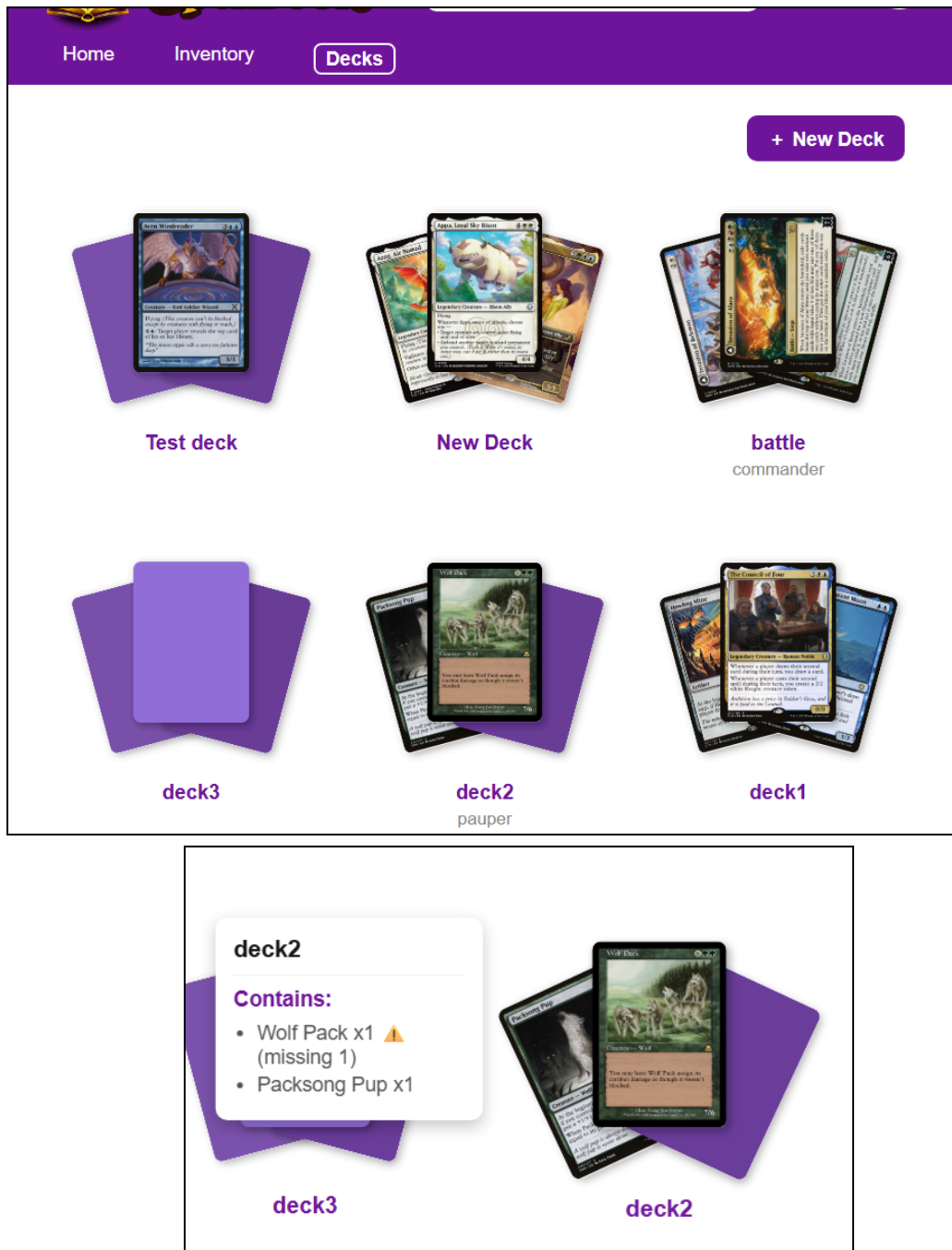


Image 6 & 7: Deck page

The Deck page displays the full contents of a specific deck when clicked from the DeckGrid. It is a fairly complex page built off of multiple components and hooks working together. The page itself is made up of a few distinct pieces, the first is a pair hooks that accesses spellbook's database using the parameter passed in the url to gather the information pertaining to the decks name and format and then find all cards associated with the given deck id. It then works to break down this data, sorting the cards by types. The page passes the data from these hooks into its display component and its mapping function. The display component breaks the data down further into specifics about each card using another hook; using this data it allows the page to generate a curve to allow the user to quickly glance at the average cost of cards in their

deck. It also includes a few subcomponents that display pieces of information as well as change the info pertaining; this is related to changing the format or commander of the deck. An important feature of the display component is the quick add feature which acts as a text field that autopopulates five results from scryfall's api as the user types; this data is given purely as a cards name and if the user selects one of the options the card will be added to the relevant deckCards table with the appropriate deck and card id. The mapping function assigns the various sorted cards into card stack items that are children of a masonry component. The end result of this process is that various stacks of cards can easily be displayed while cleanly occupying minimal space on any given screen size. Each card in a card stack has an overlay that provides simple controls for modifying the quantities of the card within the deck. At the bottom section of the deck page there are buttons for renaming the deck and deleting it, they are at the bottom to prevent an accidental misclick; the delete deck button prompts an alert to confirm that the user wants to go through with the deletion of their deck.

Login Page

The Login page is a page that directs the user to login to the site before accessing certain features. The page appears to users when they are not logged in and try to navigate to the inventory or decks page. Upon successfully signing in, they are directed to the page they originally tried to go to. The visual layout is a centered white card with a purple border and lavender color in the background. On the card is the Spellbook logo and text, a small subtitle beneath it, and then a Google sign-in button. The button has a Google icon and text, shows "Redirecting..." when busy and processing the login, and is disabled during the redirect. If the sign-in process fails, an error message is displayed beneath the button.

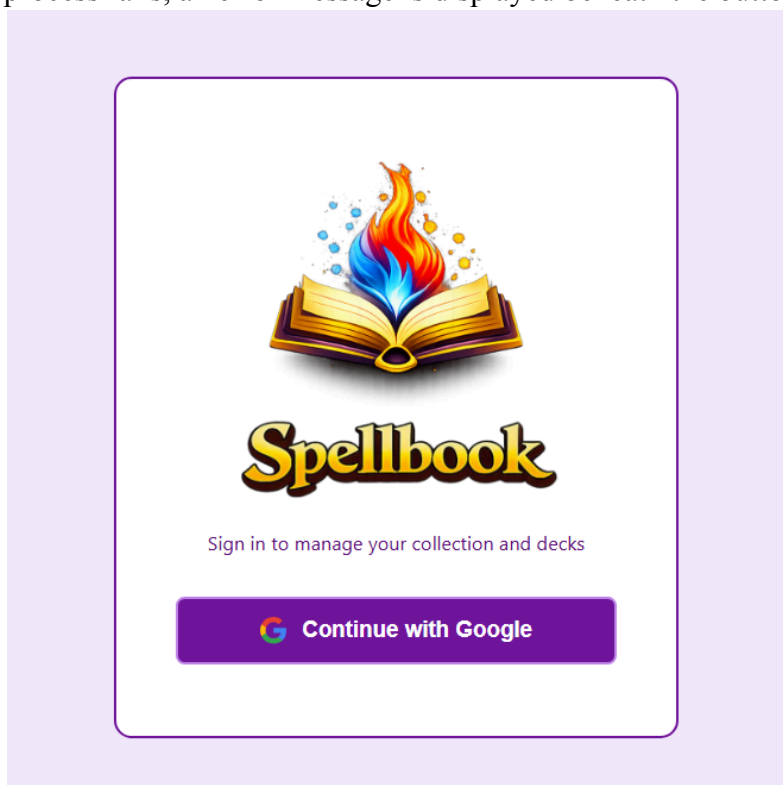


Image 8: Login screen

Backend and Database

Database Design

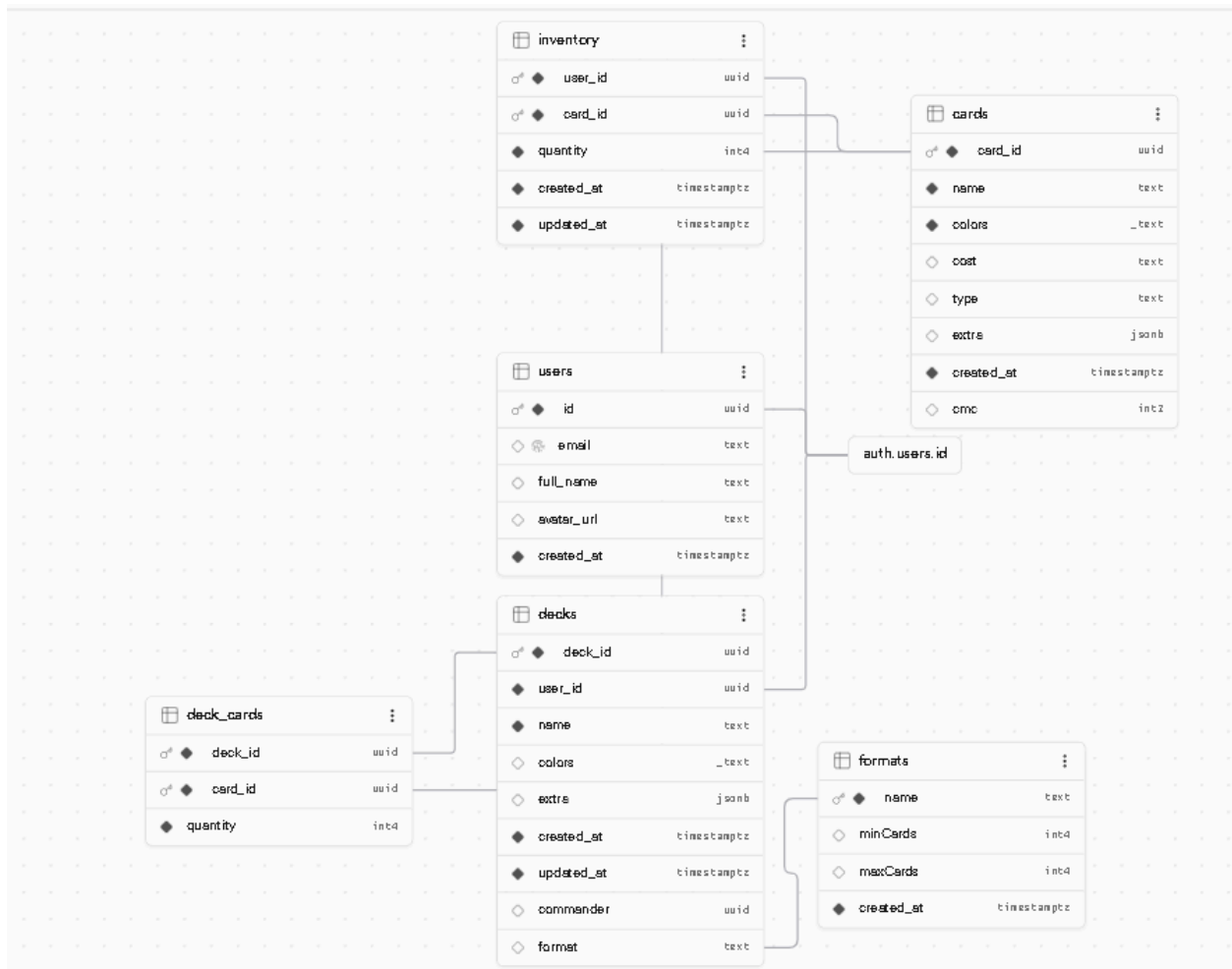


Image 9: Database tables used for Spellbook.

Spellbook’s database consists of six tables: `cards`, `decks`, `deck_cards`, `formats`, `inventory`, and `users`. With the exception of `deck_cards`, each table has a “`created_at`” column that stores the time at creation. `Decks` and `inventory` have an additional time stamp “`updated_at`” to track whenever they are modified.

“`users`” holds the data pertaining to the users of the site itself and is autopopulated from Google OAuth. Its primary key is the auto generated uuid “`id`”. The remaining columns are: `email`, `full_name`, and `avatar_url`. “`users`” is referenced from both the `decks` and `inventory` table as “`deck_id`”.

“`inventory`” consists of the following columns: `user_id`, `card_id`, `quantity`, `created_at`, and `updated_at`. `user_id` and `card_id` make up a composite key that associates the inventory item to both a user and a card. The `card_id` must exist in the `cards` table in order to be assigned to this table.

“`cards`” consists of the following columns: `card_id`, `name`, `colors`, `cost`, `type`, `extra`, `cmc`, and `created_at`. `Cost`, `type`, `cmc`, and `extra` are all optional columns and can contain null values.

The data within this table is used to track the data pertaining to a card so that spellbook can avoid querying Scryfall for everything. The card id in this table is not automatically generated but instead must match the id from the original Scryfall api call.

“decks” is made up of five necessary columns: deck_id, user_id, name, created_at, and updated_at; it also contains four optional columns: colors, extra, commander, and format. The primary key, deck_id, is a randomly generated uuid. User_id and formats are both foreign keys referencing both the users and formats table respectively. This table provides the necessary information for properly storing a deck.

“deck_cards” is a table made up of three necessary columns: deck_id, card_id, and quantity. Both deck_id and card_id make up a composite key for this table and reference both the decks and card tables respectively; a matching id must be found in the respective table for an insert to this table to work. This table is the deck itself, each item is associated with a deck and matches to a specific card while tracking the quantity contained.

“formats” has four columns: name, minCards, maxCards, created_at. Both min and max cards are optional columns. Its primary key, name, is a text element that directly correlates to the name of a given format so that legality is easy to check with Scryfall’s api. This table is used to contain simple data for supported formats.

How to Deploy or Build the Application

Downloading the Code from GitHub

The first step in building or deploying Spellbook is downloading the source code from the project’s GitHub repository. GitHub is used as the main version control system for the project, so the current version of the application is stored there. To begin, the developer should open the Spellbook GitHub repository at <https://github.com/jesselpickard/Group3>. From there, the code can either be downloaded as a ZIP file or cloned using Git. Cloning the repository is usually the better option because it allows the developer to pull future updates and keep the local copy connected to the remote project.

To clone the project, the developer should open a terminal or command prompt and run the following command:

```
git clone https://github.com/jesselpickard/Group3.git
```

After the repository has been downloaded, the developer should move into the project folder:

```
cd Group3
```

The actual Next.js application is located inside the nextinventorymanager folder, so the developer must move into that directory before installing packages or running the project:

```
cd nextinventorymanager
```

This step is important because the root GitHub repository contains the overall project, but the deployable web application is inside the `nextinventorymanager` directory. If the developer runs `build` or `install` commands from the wrong folder, the application may not compile correctly because the `package.json` file for the Next.js project will not be found.

Installing Required Software

Before the project can run locally, the developer needs to have Node.js and npm installed. Node.js allows the JavaScript application to run outside of the browser, and npm is used to install the project's required packages. Visual Studio Code is also recommended because it makes it easier to edit the files, use the terminal, and manage the GitHub repository.

Once the developer is inside the `nextinventorymanager` folder, the next step is to install all required dependencies. These dependencies are listed in the `package.json` file. They include Next.js, React, Supabase client tools, and other libraries used by the web application. The dependencies can be installed by running:

```
npm install
```

This command creates the `node_modules` folder and downloads the libraries needed to run and build the project. The `node_modules` folder should not be manually edited or uploaded separately because it is generated from the `package.json` and `package-lock.json` files.

Creating the Environment Variables

Spellbook depends on Supabase for authentication and database storage, so the application needs environment variables before it can connect to the backend. Environment variables are private project settings that store connection information and API keys outside of the main source code. This is important because API keys and backend URLs should not be hard-coded directly into the application files.

For local development, the developer should create a file named `.env.local` inside the `nextinventorymanager` folder. The file should contain the public Supabase project URL and the public Supabase anonymous key. The basic format should look like this:

```
NEXT_PUBLIC_SUPABASE_URL=https://upqoytyujlmnhbpodtuw.supabase.co
```

```
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-supabase-anon-key-here
```

The Supabase URL identifies the specific Supabase project that the website connects to. The anonymous key allows the frontend application to communicate with Supabase using the permissions allowed by the project's Row Level Security policies. The anonymous key is designed to be used by browser-based applications, but it should still be managed carefully and should not be confused with the service role key. The service role key should not be placed in the frontend application because it has much higher privileges.

Spellbook also uses the Scryfall API to retrieve Magic: The Gathering card information. Scryfall provides public card data, so the project does not require a private Scryfall API key for basic card searching and card display. Instead, the application sends requests to Scryfall endpoints to retrieve card names, images, mana costs, card types, legality information, alternate prints, and other card data.

Setting Up the Supabase Backend

The next major step is setting up the backend database in Supabase. Supabase is used for user authentication and for storing user-specific data, such as inventories and decks. Unlike Scryfall, which stores the general MTG card information, Supabase stores the information connected to Spellbook users. This includes which cards a user owns, how many copies they have, what decks they created, and which cards belong to each deck.

To set up the database, the developer should open the Supabase project dashboard and create the required tables. The main database tables for Spellbook are users, cards, inventory, decks, deck_cards, and formats. The users table stores user account information connected to authentication. The cards table stores card records that are needed by the application. The inventory table connects users to the cards they own and stores quantity information. The decks table stores the decks created by users. The deck_cards table connects decks to the cards inside them. The formats table stores deck format information, such as Commander, Standard, Modern, or Pauper if those options are being used by the application.

The database tables need to be related correctly. For example, the inventory table should connect a user to a card, while the deck_cards table should connect a deck to a card. This allows the application to know which user owns which cards and which cards belong inside each deck. These relationships are important because Spellbook is not just displaying card information from Scryfall; it is also storing personalized collection data for each logged-in user.

After the tables are created, Row Level Security policies should be enabled and configured. Row Level Security makes sure that users can only access or modify their own inventory and deck information. For example, a logged-in user should be able to add cards to their own inventory, but they should not be able to change another user's inventory. This protects the data and keeps each user's collection separate.

Setting Up Google OAuth Login

Spellbook uses Google login through Supabase authentication. To make this work, Google OAuth must be enabled in the Supabase dashboard. The developer must create or configure Google OAuth credentials in the Google Cloud Console and then copy the required Client ID and Client Secret into the Supabase authentication provider settings.

In Supabase, the developer should go to the Authentication section, open the Providers settings, and enable Google as a sign-in provider. The Google Client ID and Client Secret should then be entered into the Google provider settings. The project should also have the correct redirect URLs configured. Redirect URLs are important because after a user signs in with Google, Google and Supabase need to know where to send the user back.

For the deployed version of Spellbook, the redirect URL should match the Vercel website domain. For local testing, a localhost redirect URL may also be needed. This allows developers to test login locally before deploying the final version. If the redirect URLs are incorrect, the

login button may appear to work at first, but the user may be redirected to an error page or may not be returned to Spellbook correctly.

Running the Application Locally

After the code has been downloaded, the dependencies have been installed, the environment variables have been added, and Supabase has been configured, the project can be tested locally. The developer should make sure they are still inside the `nextinventorymanager` folder and then run:

```
npm run dev
```

This starts the Next.js development server. In most cases, the application will be available at:

```
http://localhost:3000
```

The local development version allows the developer to test the website before deploying it. This is where the developer should check that the home page loads, the Navbar displays correctly, card searching works, card information pages open, Google login works, inventory features work, and deck features work. Testing locally is important because deployment errors are easier to fix before the project is uploaded to Vercel.

If the application does not run, the developer should first check that `npm install` was completed successfully. Next, they should check that the `.env.local` file exists in the correct folder and that the Supabase environment variables are spelled correctly. In Next.js, public environment variables that are used in the browser must begin with `NEXT_PUBLIC_`. If the variable names are incorrect, the application may not be able to connect to Supabase.

Compiling the Code

Before deploying the project, the developer should run a production build locally. This checks whether the application can compile successfully outside of development mode. Development mode can sometimes hide problems that only appear during the production build, so this step is important before pushing changes to GitHub or deploying through Vercel.

The build command is:

```
npm run build
```

This command tells Next.js to compile the project for production. It checks pages, components, imports, routing, and other build-time requirements. If there is an error, the terminal will show which page or file caused the problem. For example, if a component is used but not imported, the local development server might not always show the issue immediately, but the production build will usually fail.

After the build finishes successfully, the developer can optionally run the production version locally by using:

```
npm run start
```

This starts the compiled production build. Running the application this way is closer to how the website will behave after deployment. If both `npm run build` and `npm run start` work correctly, the application is ready to be deployed.

Setting Up the Vercel Project

Vercel is used to host and deploy Spellbook as a live website. The project can be deployed by connecting the GitHub repository to Vercel. In the Vercel dashboard, the developer should create a new project and import the GitHub repository named Group3. During setup, Vercel will ask for project configuration settings.

The most important setting is the Root Directory. Since the Next.js application is inside the `nextinventorymanager` folder, the Root Directory in Vercel should be set to:

```
nextinventorymanager
```

This tells Vercel where the actual web application is located. If the root directory is left as the main repository folder, Vercel may not detect the Next.js project correctly, or it may fail to find the correct `package.json` file. Setting the root directory correctly allows Vercel to install dependencies, run the build command, and deploy the correct application.

The framework preset should be set to Next.js. The install command can usually remain `npm install`, and the build command should be `npm run build`. Vercel normally detects these settings automatically when the root directory is correct, but they should still be checked during setup.

Adding Environment Variables in Vercel

The environment variables used locally must also be added to Vercel. The `.env.local` file is only used on the developer's computer and is not automatically available in the deployed website. In the Vercel project dashboard, the developer should open the project settings and go to the Environment Variables section.

The following variables should be added:

```
NEXT_PUBLIC_SUPABASE_URL
```

```
NEXT_PUBLIC_SUPABASE_ANON_KEY
```

The `NEXT_PUBLIC_SUPABASE_URL` value should be:

```
https://upqoytyujlmnhbpdtuw.supabase.co
```

The `NEXT_PUBLIC_SUPABASE_ANON_KEY` value should be copied from the Supabase project settings. After these variables are added, the project should be redeployed so the build can use the updated settings.

If these environment variables are missing, the deployed website may load visually, but Supabase features such as login, inventory, and decks may not work. This is because the frontend will not know which Supabase project to connect to.

Deploying the Website

After the Vercel project is connected to GitHub and the environment variables are added, the application can be deployed. Vercel will usually deploy automatically when changes are pushed to the connected GitHub branch. This means the team can update the code in GitHub, and Vercel will rebuild the website using the latest version.

The deployment process starts when Vercel installs the project dependencies, runs the build command, and publishes the finished website. If the build succeeds, Vercel provides a live website URL. If the build fails, Vercel shows a build log that explains what went wrong. Common problems include missing imports, incorrect file paths, missing environment variables, or the wrong root directory.

Once the deployment succeeds, the developer should open the live Vercel website and test the main features. The home page should display cards from Scryfall. The search and filter features should return card results. The card info page should open when a card is selected. Google login should redirect correctly and return the user to Spellbook. The inventory page should allow authenticated users to add and remove cards. The decks page should allow users to view, create, and manage decks.

Final User-Ready Application

At the end of the deployment process, Spellbook should be available as a user-ready website hosted through Vercel. Users should be able to visit the deployed site in a browser without installing anything locally. The frontend is served by Vercel, the user authentication and database are handled by Supabase, and Magic: The Gathering card data is retrieved from the Scryfall API.

The final build and deployment process can be summarized as downloading the code from GitHub, moving into the `nextinventorymanager` folder, installing dependencies with `npm install`, setting up the Supabase database and authentication, adding the required environment variables, testing locally with `npm run dev`, compiling the project with `npm run build`, connecting the GitHub repository to Vercel, setting the Vercel root directory to `nextinventorymanager`, adding environment variables in Vercel, and deploying the finished website.

This process creates a complete web application where users can log in with Google, search Magic: The Gathering cards, view detailed card information, manage their inventory, and organize cards into decks.

Known Issues

The application currently has multiple known issues and limitations that could be fixed during this semester. One issue is the fetch limit of data from the Scryfall API. To avoid

overloading the number of queries we request, the application has a limit on how many it can run in a single pull. This prevents the user from pulling the entire available dataset at once. But this should be a big issue, since most of the time, when a user searches for a card, they are looking for something very specific. A fix for this could be to implement pagination, pulling multiple smaller requests into a single large data pool. Another fix could be to create our own dataset and rely on it.

The next issue we have is the double-sided land cards in the Jurassic Park set. When adding this card to your deck, it does nothing. You can view it on your inventory, search, and card info pages. But it will not allow the user to add to or view the deck. Since a double-sided land works the same as a regular land, it doesn't cause that big of an issue. The only issue is that a user may want to keep track of that exact card in their deck and of land in general. The fix would be to identify what is causing this error, but due to time constraints, the team decided to look past it.

Lastly, multiple language support is an issue that we have an answer to, but not enough time to solve. It wasn't thought about until the end of the project. But so far, the only supported language is English. If you change your browser's language, most of the app's UI will be translated into your preferred language. When searching for cards in another language, the user can enter lang: and the card's name in their language to have that card pop up. We could make lang: the default, but hide it from the user. So that when they search for a card, it automatically searches the card in the preferred language without having to type lang: every time. We could also implement a button that changes the site to that language. The only real drawback is that we would need to create multiple logos, and Scryfall only supports a limited number of languages.